ELTE IK - Computer Science BSc

Final Exam Topics

13.1 Logic

Note: This document was translated from Hungarian to English by AI. Please verify the information against the original Hungarian source before relying on it.

Logic

Propositional calculus and first-order predicate calculus: syntax, semantics, equivalent transformations, the concept of semantic consequence, resolution.

1 Logic

1.1 Basic concepts

The subject of logic is the examination of the human thinking process and the search for, or creation of, correct forms of thinking.

Concepts:

1. **Statement**: A declarative sentence whose logical value (truth) is decidable, true or false in any context. We say that a statement is true if its information content corresponds to reality (the facts), and false in the opposite case.

   The declarative sentences used in everyday speech are most often not statements, since the context is also part of the meaning of the sentence: time, state of the environment, a certain level of general knowledge, etc. (e.g. "the sun was shining at 8 a.m. this morning" is not a statement, but e.g. "every even number is divisible by 2" is a statement).

2. **Truth value**: The set of truth values is $\mathbb{L} = \{true, false\}$.

3. **Form of thinking**: By a form of thinking we mean a pair (F, A) where A is a statement, and $F = \{A_1, A_2, ..., A_n\}$ is a set of statements.

   The form of thinking is correct if in every case when every statement of F is true, A is also true.

1.2 Propositional calculus

1.2.1 The syntax of propositional logic

The alphabet of propositional logic

The alphabet of propositional logic is $V_0 = V_v \cup \{(,)\} \cup \{\neg, \wedge, \vee, \supset\}$, where V_v is the set of propositional variables. So V_0 contains the propositional variables, the parentheses, and the signs of the logical operations.

The language of propositional logic

The language of propositional logic ($\mathcal{L}_0$) consists of propositional-logic formulas, which are produced as follows:

1. Every propositional variable is a propositional-logic formula. These are the so-called prime formulas (or atomic formulas).
2. If A is a propositional-logic formula, then so is $\neg A$.
3. If A and B are propositional-logic formulas, then $(A \wedge B)$, $(A \vee B)$ and $(A \supset B)$ are also propositional-logic formulas.
4. Every propositional-logic formula is produced by the finitely many times application of rules 1-3.

Literal: If X is a propositional variable, then the formulas X and $\neg X$ are literals whose base is X.

Direct subformula:

1. A prime formula has no direct subformula.
2. The direct subformula of $\neg A$ is A.
3. The direct subformulas of $A \circ B$ ($\circ$ is one of the binary connectives $\wedge, \vee, \supset$) are A (left-hand) and B (right-hand).

Subformula: Let $A \in \mathcal{L}_0$ be a propositional-logic formula. Then the set of subformulas of A is the narrowest set that

1. has A as an element, and
2. if the formula C is an element, then the direct subformulas of C are also elements.

Structure tree: The structure tree of a formula C is a finite ordered tree whose vertices are formulas,

1. its root is C,
2. the vertex $\neg A$ has exactly one child, A,
3. the vertex $A \circ B$ has exactly two children, respectively the formulas A and B,
4. its leaves are prime formulas.

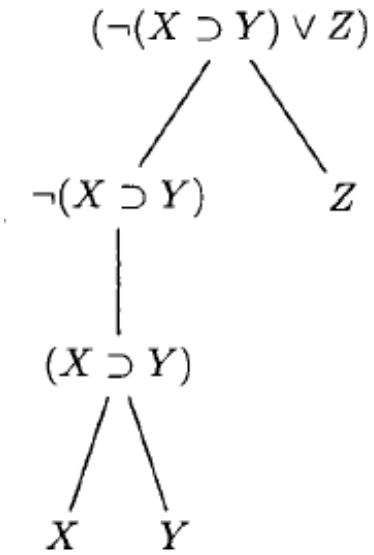

Figure 1: Example of a structure tree.

Logical complexity: The logical complexity of a formula is the number of logical connectives found in it.

Scope of an operation: The scope of an operation is, among the subformulas of the formula, the subformula of smallest logical complexity in which the given operation occurs.

Main logical connective: The main logical connective of a formula is the connective whose scope is the formula itself.

Precedence: The precedence of the logical connectives in decreasing order is the following: $\neg, \wedge, \vee, \supset$.

Based on the definitions, it is unambiguous in a *fully parenthesized formula* what the scope of the logical connectives is and what the main logical connective is. Now we show in which cases and which parentheses bounding which subformulas can be omitted in a formula so that the scope of the logical connectives does not change. Among the subformulas, the prime formulas and the negation formulas have no outer parenthesis pair, so we only have to decide about subformulas of the form $(A \circ B)$ whether $A \circ B$ can be written instead. We always start the omission of parentheses with the omission of the outer parenthesis pair of the formula (if there is one). Then, once we have examined the question of outer-parenthesis omission in a subformula, after that we deal with the outer parentheses of the direct subformulas of this subformula. Two cases are possible:

1. The subformula is a negation formula, in which the outer parentheses of the direct subformula of the form $(A \circ B)$ cannot be omitted.
2. The subformula is a formula of the form $(A \bullet B)$ or $A \bullet B$, in whose direct subformulas A and B the fate of the outer parentheses has to be decided. If the formula A is of the form $A_1 \circ A_2$, then the outer parenthesis pair of A can be omitted if $\circ$ has higher precedence than $\bullet$. If the formula B is of the form $B_1 \circ B_2$, then the outer parenthesis pair of B can be omitted if $\circ$ has higher or equal precedence than $\bullet$.
3. If some direct subformula of a formula of the form $(A \wedge B)$ or $A \wedge B$ is likewise a conjunction, or some direct subformula of a formula of the form $(A \vee B)$ or $A \vee B$ is likewise a disjunction, then the outer parenthesis pair can be omitted from such subformulas.

Formula chains: Taking into account the conventions on the omission of parentheses, we can also obtain so-called conjunction, disjunction, and implication formula chains. Their form is $A_1 \wedge ... \wedge A_n$, $A_1 \vee ... \vee A_n$, or $A_1 \supset ... \supset A_n$. The main logical connective of these chain formulas we will determine with the following parenthesization convention: $(A_1 \wedge (A_2 \wedge ... \wedge (A_{n-1} \wedge A_n)...))$, $(A_1 \vee (A_2 \vee ... \vee (A_{n-1} \vee A_n)...))$, or $(A_1 \supset (A_2 \supset ... \supset (A_{n-1} \supset A_n)...))$

1.2.2 The semantics of propositional logic

Interpretation: By an interpretation of $\mathcal{L}_0$ we mean a function $\mathcal{I} : V_v \to \mathbb{L}$ that unambiguously assigns a truth value to every propositional variable.

Boolean valuation: The *Boolean valuation in interpretation* $\mathcal{I}$ of formulas in $\mathcal{L}_0$ is the following function $\mathcal{B}_\mathcal{I} : \mathcal{L}_0 \to \mathbb{L}$:

1. if A is a prime formula, then $\mathcal{B}_\mathcal{I}(A) = \mathcal{I}(A)$,
2. let $\mathcal{B}_\mathcal{I}(\neg A)$ be $\neg \mathcal{B}_\mathcal{I}(A)$,
3. let $\mathcal{B}_\mathcal{I}(A \wedge B)$ be $\mathcal{B}_\mathcal{I}(A) \wedge \mathcal{B}_\mathcal{I}(B)$,
4. let $\mathcal{B}_\mathcal{I}(A \vee B)$ be $\mathcal{B}_\mathcal{I}(A) \vee \mathcal{B}_\mathcal{I}(B)$,
5. let $\mathcal{B}_\mathcal{I}(A \supset B)$ be $\mathcal{B}_\mathcal{I}(A) \supset \mathcal{B}_\mathcal{I}(B)$,

Basis: A fixed order of the propositional variables of the formula.

Semantic tree: The different interpretations of a formula can be illustrated with the help of a semantic tree. The semantic tree is a binary tree whose i-th level ($i >= 1$) belongs to the i-th propositional variable of the basis, and from every vertex two edges start, one labeled with the propositional variable assigned to the level, the other with its negation. For the propositional variable X, let the label X mean that X is *true* in the given interpretation, and the label $\neg X$ that it is *false* in the given interpretation. Every branch of the semantic tree represents a possible interpretation. For a formula of n variables every branch is n long, and the tree has 2^n branches and contains all possible interpretations.

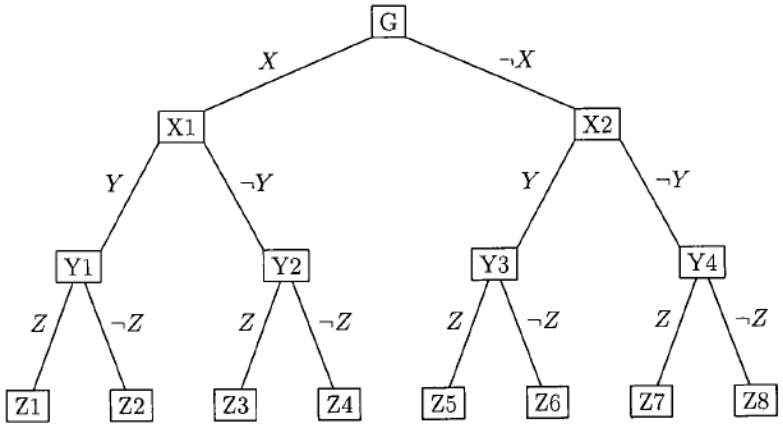

Figure 2: The semantic tree of the formula containing the propositional variables X, Y, Z.

Truth table: The truth table of a formula of n variables is a table consisting of $n + 1$ columns and 2^n rows. In the header of the table, to the i-th column ($1 <= i <= n$) the i-th propositional variable of the formula's basis, and to the $(n+1)$-th column the formula itself is assigned. In the first n columns, for the individual rows, we give respectively the different interpretations of the formula, then in the column of the formula we write into every row the truth value of the formula – obtained by the Boolean valuation in the interpretation belonging to the row.

The truth table of the logical operations:

X	Y	$\neg X$	$X \wedge Y$	$X \vee Y$	$X \supset Y$
T	T	F	T	T	T
T	F	F	F	T	F
F	T	T	F	T	T
F	F	T	F	F	T

True set, false set: The true set (A^i) of a formula A is the set of those interpretations on which the truth valuation of the formula is true. And the false set (A^h) of the formula A is the set of those interpretations for which the truth valuation of the formula is false.

Truth valuation function: A function that assigns to every formula its true set (φA^i) or its false set (φA^h).

Let A be an arbitrary propositional-logic formula. Let us determine for A the φA^i and φA^h conditions relating to its interpretations as follows:

1. If A is a prime formula, the condition φA^i is satisfied exactly by those interpretations $\mathcal{I}$ for which $\mathcal{I}(A) = true$, and the condition φA^h exactly by those for which $\mathcal{I}(A) = false$.
2. The conditions $\varphi(\neg A)^i$ hold exactly when the conditions φA^h hold.
3. The conditions $\varphi(A \wedge B)^i$ hold exactly when the conditions φA^i and φB^i hold simultaneously.

4. The conditions $\varphi(A \vee B)^i$ hold exactly when the conditions φA^i or φB^i hold.

5. The conditions $\varphi(A \supset B)^i$ hold exactly when the conditions φA^h or φB^i hold.

Theorem: For any propositional-logic formula A, the conditions φA^i are satisfied exactly by the interpretations in A^i.

Truth-valuation tree: The interpretations of a formula A satisfying the φA^i or φA^h conditions can be illustrated with the help of the truth-valuation tree. We produce the truth-valuation tree using the structure tree of the formula. To the root we assign which truth-valuation conditions of A for which truth value we are looking for, then under the root the direct subformulas of A get placed with the appropriate condition prescription, as follows:

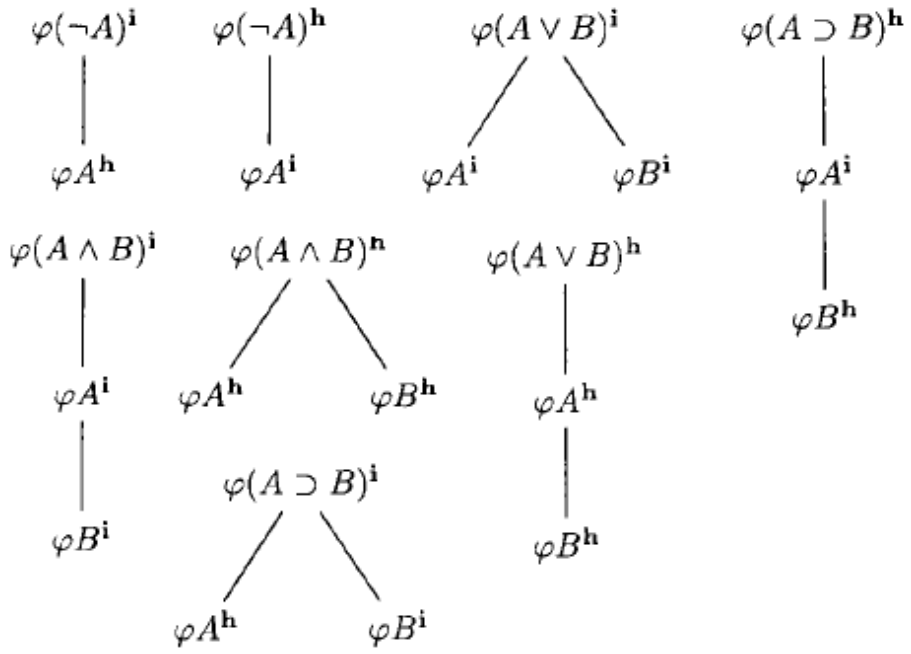

Figure 3: Condition prescriptions of the truth-valuation tree.

After this we assign to the root the ✓(processed) mark. We continue the procedure recursively until, on a branch, the unprocessed formulas

- (a) all become propositional variables, or
- (b) a mutually contradicting prescription appears for the same formula.

In case (a) the n-tuples containing the truth values, fixed on the branch, of the propositional variables occurring on the branch are all elements of the true set of the formula in the case of a φA^i root, and of the false set of the formula in the case of a φA^h root.

In case (b) no such truth-value n-tuple is produced.

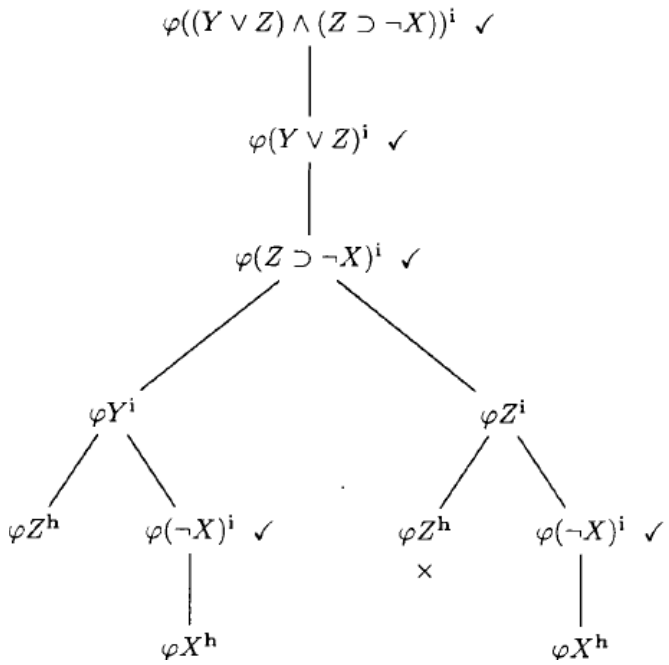

Figure 4: The truth-valuation tree of the formula $(Y \vee Z) \wedge (Z \supset \neg X)$.

In the above example the true set of the formula, based on the truth-valuation tree, is: $\{(T,T,F),(F,T,T),(F,T,F),(F,F,T)\}$

Extended truth table: To facilitate the computation of the truth value of the formula in a truth table, the extended truth table was introduced. In the extended truth table, besides the columns assigned to the propositional variables and to the formula, the columns belonging to the subformulas of the formula also appear, in turn. Essentially the subformulas appearing in the structure tree are listed.

X	Y	Z	$Y \vee Z$	$\neg X$	$Z \supset \neg X$	$(Y \vee Z) \wedge (Z \supset \neg X)$
i	i	i	i	h	h	h
i	i	h	i	h	i	i
i	h	i	i	h	h	h
i	h	h	h	h	i	h
h	i	i	i	i	i	i
h	i	h	i	i	i	i
h	h	i	i	i	i	i
h	h	h	h	i	i	h

Figure 5: The extended truth table of the formula $(Y \vee Z) \wedge (Z \supset \neg X)$.

Satisfiability of a formula, its model: A propositional-logic formula A is *satisfiable* if there exists an interpretation $\mathcal{I}$ for which $\mathcal{I} \models_0 A$, that is, the Boolean valuation $\mathcal{B}_\mathcal{I}$ assigns a true value to A. Such an interpretation is called a *model* of A. If A has no model, then we say that it is *unsatisfiable*.

If in the truth table of A there is a row in which a true value appears in the column of the formula, then the formula is satisfiable, otherwise unsatisfiable. Likewise, if φA^i is not empty, then it is satisfiable, otherwise unsatisfiable.

Propositional-logic law, tautology: A propositional-logic formula A is a *propositional-logic law* or otherwise a *tautology* if every interpretation of $\mathcal{L}_0$ is a model of A. (notation: $\models_0 A$)

Decision problem: We call the following tasks decision problems:

1. Decide for an arbitrary formula whether it is a tautology!

2. Decide for an arbitrary formula whether it is unsatisfiable!

Tautologically equivalent formulas: The propositional-logic formulas A and B are *tautologically equivalent* (notation: $A \sim_0 B$) if in every interpretation $\mathcal{I}$ of $\mathcal{L}_0$, $\mathcal{B}_\mathcal{I}(A) = \mathcal{B}_\mathcal{I}(B)$.

Satisfiability of a formula set, its model: An arbitrary set Γ of formulas of $\mathcal{L}_0$ is satisfiable if there is an interpretation $\mathcal{I}$ of $\mathcal{L}_0$ for which: $\forall A \in \Gamma : \mathcal{I} \models_0 A$. Such an interpretation $\mathcal{I}$ is a model of Γ. If Γ has no model, then Γ is unsatisfiable.

Lemma: The models of a formula set $\{A_1, A_2, ..., A_n\}$ are exactly those interpretations $\mathcal{I}$ that are models of the formula $A_1 \wedge A_2 \wedge ... \wedge A_n$. Consequently $\{A_1, A_2, ..., A_n\}$ is unsatisfiable exactly when the formula $A_1 \wedge A_2 \wedge ... \wedge A_n$ is unsatisfiable.

Semantic consequence: Let Γ be an arbitrary set of propositional-logic formulas, B an arbitrary formula. We say that the formula B is a *tautological consequence* of the formula set Γ (notation: $\Gamma \models_0 B$) if every interpretation that is a model of Γ is also a model of B. The formulas in Γ are called condition formulas or premises, and the formula B the consequence formula (conclusion).

Theorem: Let Γ be an arbitrary set of propositional-logic formulas, A, B, C arbitrary propositional-logic formulas. If $\Gamma \models_0 A$, $\Gamma \models_0 B$ and $\{A, B\} \models_0 C$, then $\Gamma \models_0 C$.

Theorem: Let $A_1, A_2, ..., A_n, B$ be arbitrary propositional-logic formulas. $\{A_1, A_2, ..., A_n\} \models_0 B$ exactly when the formula set $\{A_1, A_2, ..., A_n, \neg B\}$ is unsatisfiable, that is, the formula $A_1 \wedge A_2 \wedge ... \wedge A_n \wedge \neg B$ is unsatisfiable.

Theorem: Let $A_1, A_2, ..., A_n, B$ be arbitrary propositional-logic formulas. $\{A_1, A_2, ..., A_n\} \models_0 B$ exactly when $\models_0 A_1 \wedge A_2 \wedge ... \wedge A_n \supset B$.

Equivalent transformations

Concepts:

1. A prime formula (propositional variable), or its negation, is by a common name called a *literal*. The prime formula is the *base of the literal*. A literal is in certain cases also called a *unit conjunction* or *unit disjunction* (*unit clause*).

2. An *elementary conjunction* is the unit conjunction, or the conjunction of literals with different bases ($\wedge$ connection between the literals). An *elementary disjunction* or *clause* is the unit disjunction and the disjunction of literals with different bases ($\vee$ connection between the literals). An elementary conjunction, or elementary disjunction, is *complete* with respect to an n-variable logical operation if all n propositional variables are the base of some of its literals.

3. By *disjunctive normal form* (DNF) we mean the disjunction of elementary conjunctions. By *conjunctive normal form* (CNF) we mean the conjunction of elementary disjunctions. We speak of *canonical* disjunctive or conjunctive normal forms (CDNF, or CCNF) if the elementary conjunctions, or elementary disjunctions, appearing in them are complete.

Producing the CDNF, CCNF describing an arbitrary logical operation: Let $b : \mathbb{L}^n \to \mathbb{L}$ be an n-variable logical operation. Let us give the operation table of b. In the header of the first n columns let us write the propositional variables X_1, X_2, ... X_n.

Producing the CDNF describing b:

1. Let us choose those rows in the operation table where b assigns a *true* value to the given truth-value n-tuple. Let these rows be respectively $s_1, s_2, ..s_r$. To every such row let us assign a complete elementary conjunction $X'_1 \wedge X'_2 \wedge ... \wedge X'_n$ such that the literal X'_j is X_j or $\neg X_j$ according to whether in this row X_j has truth value *true* or *false*. Let the complete elementary conjunctions thus obtained be respectively $k_{s_1}, k_{s_2}, ..k_{s_r}$.

2. From the complete elementary conjunctions thus obtained let us make a disjunction chain formula: $k_{s_1} \vee k_{s_2} \vee ... \vee k_{s_r}$. This formula will be the canonical disjunctive normal form (CDNF) of the operation b.

X	Y	Z	b		a teljes elemi konjunkciók
i	i	i	h		
i	i	h	i	$\star$	$X \wedge Y \wedge \neg Z$
i	h	i	h		
i	h	h	i	$\star$	$X \wedge \neg Y \wedge \neg Z$
h	i	i	i	$\star$	$\neg X \wedge Y \wedge Z$
h	i	h	h		
h	h	i	i	$\star$	$\neg X \wedge \neg Y \wedge Z$
h	h	h	i	$\star$	$\neg X \wedge \neg Y \wedge \neg Z$

Figure 6: The operation table of a three-variable logical operation b and the produced complete elementary conjunctions.

The canonical disjunctive normal form of the operation b in the above example is the following formula: $(X \wedge Y \wedge \neg Z) \vee (X \wedge \neg Y \wedge \neg Z) \vee (\neg X \wedge Y \wedge Z) \vee (\neg X \wedge \neg Y \wedge Z) \vee (\neg X \wedge \neg Y \wedge \neg Z)$.

Producing the CCNF describing b:

1. Let us choose those rows in the operation table where b assigns a *false* value to the given truth-value n-tuple. Let these rows be respectively $s_1, s_2, ..s_r$. To every such row let us assign a complete elementary disjunction $X'_1 \vee X'_2 \vee ... \vee X'_n$ such that the literal X'_j is X_j or $\neg X_j$ according to whether in this row X_j has truth value *false* or *true*. Let the complete elementary disjunctions thus obtained be respectively $d_{s_1}, d_{s_2}, ..d_{s_r}$.

2. From the complete elementary disjunctions thus obtained let us make a conjunction chain formula: $d_{s_1} \wedge d_{s_2} \wedge ... \wedge d_{s_r}$. This formula will be the canonical conjunctive normal form (CCNF) of the operation b.

X	Y	Z	b		a teljes elemi diszjunkciók
i	i	i	h	$\star$	$\neg X \vee \neg Y \vee \neg Z$
i	i	h	i		
i	h	i	h	$\star$	$\neg X \vee Y \vee \neg Z$
i	h	h	i		
h	i	i	i		
h	i	h	i		
h	h	i	h	$\star$	$X \vee Y \vee \neg Z$
h	h	h	i		

Figure 7: The operation table of a three-variable logical operation b and the produced complete elementary disjunctions.

The canonical conjunctive normal form of the operation b in the above example is the following formula: $(\neg X \vee \neg Y \vee \neg Z) \wedge (\neg X \vee Y \vee \neg Z) \wedge (X \vee Y \vee \neg Z)$.

Simplification of CNF, DNF: By the logical complexity of a propositional-logic formula we meant the number of logical connectives appearing in the formula. Among formulas describing the same logical operation, we regard as simpler the one whose logical complexity is smaller (that is, which contains fewer logical connectives).

Let X be a propositional variable, k an elementary conjunction not containing X, d an elementary disjunction not containing X. Then, applying the simplification rules

- (a) $(X \wedge k) \vee (\neg X \wedge k) \sim_0 k$ and
- (b) $(X \vee d) \wedge (\neg X \vee d) \sim_0 d$

we can rewrite conjunctive and disjunctive normal forms into a simpler form.

Classical Quine–McCluskey algorithm for simplifying CDNF:

1. Let us list all the complete elementary conjunctions appearing in the CDNF in the list L_0, $j := 0$.
2. We examine all the possible elementary-conjunction pairs appearing in L_j, whether the simplification rule (a) is applicable to them. If yes, then we mark the two chosen conjunctions with ✓, and write the resulting conjunction into the list L_{j+1}. Those elementary conjunctions that during the examination of L_j do not get marked were not simplifiable, so they get into the simplified disjunctive normal form.
3. If the conjunction list L_{j+1} is not empty, then $j := j + 1$. Let us perform step 2 again.
4. From the elementary conjunctions obtained during the algorithm but not marked, let us make a disjunction chain formula. Thus we obtain a simplified DNF logically equivalent to the original CDNF.

Resolution

Let $A_1, A_2, ..., A_n, B$ be arbitrary propositional-logic formulas. We want to prove that $\{A_1, A_2, ..., A_n\} \models_0 B$, which is equivalent to $\{A_1, A_2, ..., A_n, \neg B\}$ being unsatisfiable. Let us rewrite the formulas of this latter formula set into CNF form! Then we obtain the formula set $\{CNF_{A_1}, CNF_{A_2}, ..., CNF_{A_n}, CNF_{\neg B}\}$, which is unsatisfiable exactly when the set of clauses appearing in the formulas of the set is unsatisfiable.

According to the simplification rule for clauses, if X is a propositional variable, and C is a clause not containing X, then $(X \vee C) \wedge (\neg X \vee C) \sim_0 C$. The clause equivalent to the conjunction of the unit clauses X and $\neg X$ (we say that X and $\neg X$ are a complementary literal pair) that is simpler and contains not a single literal is the empty clause, which we denote by the $\square$ sign and which by definition has the truth value false in every interpretation.

Let now C_1 and C_2 be clauses that contain exactly one complementary literal pair, that is, $C_1 = C_1' \vee L_1$ and $C_2 = C_2' \vee L_2$, where L_1 and L_2 are the single complementary literal pair (C_1' and C_2' can also be empty clauses). It is clear that if in the two clauses there are also literals besides the complementary literal pair, and these are not all identical, the applicability condition of the simplification rule does not hold.

Theorem: If $C_1 = C_1' \vee L_1$ and $C_2 = C_2' \vee L_2$, where L_1 and L_2 are a complementary literal pair, then $\{C_1, C_2\} \models_0 C_1' \vee C_2'$

Resolvent: Let C_1 and C_2 be clauses that contain exactly one complementary literal pair, that is, $C_1 = C_1' \vee L_1$ and $C_2 = C_2' \vee L_2$, where L_1 and L_2 are the complementary literal pair; the clause $C_1' \vee C_2'$ is called the *resolvent* of the clause pair (C_1, C_2) (or of the formula $C_1 \vee C_2$). If $C_1 = L_1$ and $C_2 = L_2$ (that is, C_1' and C_2' are empty clauses), their resolvent is the empty clause ($\square$). The activity whose result is the resolvent is *resolving*.

	klózpár	rezolvens
(a)	$(X \vee Y,\ \neg Y \vee Z)$	$X \vee Z$
(b)	$(X \vee \neg Y,\ \neg Y \vee Z)$	nincs: mindkét azonos alapú literál negált
(c)	$(X \vee \neg Y,\ Z \vee \neg V)$	nincs: nincs azonos alapú literál
(d)	$(\neg X \vee \neg Y,\ X \vee Y \vee Z)$	nincs: két komplemens literálpár van
(e)	$(X,\ \neg X)$	$\square$

Figure 8: Examples of the resolvability and resolvent of clause pairs.

Theorem: If the clause C is the resolvent of the clause pair (C_1, C_2), then those interpretations $\mathcal{I}$ cannot satisfy the clause set $\{C_1, C_2\}$ in which the truth value of C is false, that is, $\mathcal{B}_{\mathcal{I}}(C) = false$.

Resolution derivation: A resolution derivation of the clause C from a clause set S is a finite clause sequence $k_1, k_2, ..., k_m (m \geq 1)$ where for every $j = 1, 2, ..., m$

1. either $k_j \in S$,
2. or there is some $1 \leq s, t \leq j$ such that k_j is the resolvent of the clause pair (k_s, k_t),

and the last term of the clause sequence, k_m, is exactly the clause C.

By our convention the decision problem of the resolution calculus is whether $\square$ is derivable from S. The goal of the resolution derivation is therefore the derivation of $\square$ from S. That $\square$ is derivable from S can also be expressed as: there exists a resolution refutation of S.

Example: Let us try to derive $\square$ from the clause set $S = \{\neg X \vee Y, \neg Y \vee Z, X \vee Z, \neg V \vee Y \vee Z, \neg Z\}$. The derivation can be started from any clause in S.

1.	$\neg V \vee Y \vee Z$	[$\in S$]
2.	$\neg Z$	[$\in S$]
3.	$\neg V \vee Y$	[1, 2 rezolvense]
4.	$\neg Y \vee Z$	[$\in S$]
5.	$\neg Y$	[2, 4 rezolvense]
6.	$\neg V$	[3, 5 rezolvense]
7.	$X \vee V$	[$\in S$]
8.	X	[6, 7 rezolvense]
9.	$\neg X \vee Y$	[$\in S$]
10.	Y	[8, 9 rezolvense]
11.	$\square$	[5, 10 rezolvense]

Figure 9: Resolution derivation of $\square$ from S.

Lemma: Let S be an arbitrary clause set. In the case of a resolution derivation from S, any clause derived from S is a tautological consequence of S.

Soundness of the resolution calculus: The resolution calculus is *sound*, that is, for any clause set S, if $\square$ is derivable from S, then S is *unsatisfiable*.

Completeness of the resolution calculus: The resolution calculus is *complete*, that is, for any finite, unsatisfiable clause set S, $\square$ is derivable from S.

Derivation tree: The structure of a resolution derivation can be illustrated with the help of a *derivation tree*. The vertices of the derivation tree are clauses. From two vertices an edge leads into a third, common vertex exactly when that is the resolvent of the two clauses.

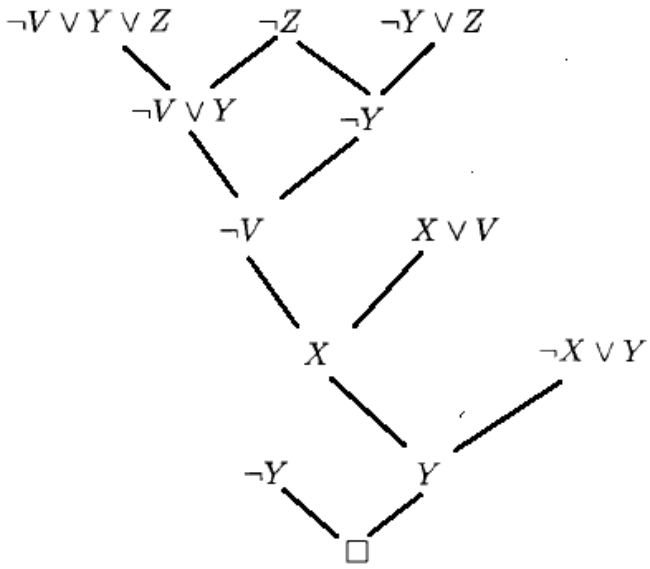

Figure 10: The derivation tree of the previous example.

Resolution strategies:

- *Linear resolution*: A linear resolution derivation from a clause set S is a resolution derivation $k_1, l_1, k_2, l_2, ..., k_{m-1}, l_{m-1}, k_m$ in which for every $j = 2, 3, ..., m$, k_j is the resolvent of the clause pair (k_{j-1}, l_{j-1}). The clauses k_j we call central clauses, and the clauses l_j side clauses.

  Any resolution derivation can be rewritten into a linear one, that is, the linear resolution calculus is complete.

- *Linear input resolution*: A linear input resolution derivation from a clause set S is a linear resolution derivation $k_1, l_1, k_2, l_2, ..., k_{m-1}, l_{m-1}, k_m$ in which for every $j = 1, 2, ..., m-1$, $l_j \in S$, that is, in the linear input resolution derivation the side clauses are elements of S.

  The linear input resolution strategy is not complete, but a formula class can be given for which it is. The clauses containing at most one negated literal are called Horn clauses, and the Horn formulas are those formulas whose conjunctive normal form is a conjunction of Horn clauses. The linear input resolution strategy is complete in the case of Horn formulas.

1.3 Predicate calculus

1.3.1 Syntax of first-order logical languages

The alphabet of a first-order logical language contains logical and non-logical symbols, as well as separators. The non-logical symbol set can be given in the form $< Srt, Pr, Fn, Cnst >$, where:

1. Srt is a nonempty set, its elements symbolize sorts,
2. Pr is a nonempty set, its elements are predicate symbols,
3. the elements of the set Fn are function symbols,
4. and $Cnst$ is the set of constant symbols.

The signature of the alphabet $< Srt, Pr, Fn, Cnst >$ is a triple $< \nu_1, \nu_2, \nu_3 >$, where

1. to every predicate symbol $P \in Pr$, ν_1 assigns the shape of the predicate symbol, that is, the sort sequence $(\pi_1, \pi_2, ..., \pi_k)$,
2. to every function symbol $f \in Fn$, ν_2 assigns the shape of the function symbol, that is, the sort sequence $(\pi_1, \pi_2, ..., \pi_k, \pi)$ and
3. to every constant symbol $c \in Cnst$, ν_3 assigns the shape of the constant symbol, that is, (π)

($k > 0$ and $\pi_1, \pi_2, ..., \pi_k, \pi \in Srt$).

Logical signs are the logical connectives also used in propositional logic, as well as the universal ($\forall$) and existential ($\exists$) quantifiers and the individual variables of different sorts. In a first-order language alphabet, to every sort $\pi \in Srt$ belongs a countably infinite system $v_1^\pi, v_2^\pi, ...$ of symbols; these symbols we call variables of sort π. Separators are the opening and closing parentheses, and the comma.

In first-order logical languages the separators and the logical signs are always the same, but the set of non-logical signs, and their signature, can differ significantly from language to language. Therefore we always give the quadruple $< Srt, Pr, Fn, Cnst >$ and its $< \nu_1, \nu_2, \nu_3 >$ signature when we refer to the alphabet of a first-order logical language. Its notation is $V[V_\nu]$, where V_ν gives the quadruple $< Srt, Pr, Fn, Cnst >$ with signature $< \nu_1, \nu_2, \nu_3 >$.

Terms: The set of terms over the alphabet $V[V_\nu]$ is $\mathcal{L}_t[V_\nu]$, which has the following properties:

1. Every variable and constant of sort $\pi \in Srt$ is a term of sort π.
2. If the function symbol $f \in Fn$ is of the form $(\pi_1, \pi_2, ..., \pi_k, \pi)$ and $t_1, t_2, ..., t_k$ are terms – respectively of sorts $\pi_1, \pi_2, ..., \pi_k$ –, then $f(s_1, s_2, ..., s_k)$ is a term of sort π.
3. Every term is produced by the finitely many times application of rules 1-2.

Formulas: The set of first-order formulas over the alphabet $V[V_\nu]$ is $\mathcal{L}_f[V_\nu]$, which has the following properties:

1. If the predicate symbol $P \in Pr$ is of the form $(\pi_1, \pi_2, ..., \pi_k)$ and $t_1, t_2, ..., t_k$ are terms – respectively of sorts $\pi_1, \pi_2, ..., \pi_k$ –, then the word $P(t_1, t_2, ..., t_k)$ is a first-order formula. The formulas thus obtained are called atomic formulas.
2. If S is a first-order formula, then so is $\neg S$.
3. If S and T are first-order formulas and $\circ$ is a binary logical connective, then $(S \circ T)$ is also a first-order formula.
4. If S is a first-order formula, Q is a quantifier ($\forall$ or $\exists$) and x is an arbitrary variable, then QxS is also a first-order formula. The formulas thus obtained are called quantified formulas, the formulas of the form $\forall xS$ are universally quantified formulas, and the formulas of the form $\exists xS$ are existentially quantified formulas. In quantified formulas Qx is the prefix of the formula, and S its matrix.
5. Every first-order formula is produced by the finitely many times application of rules 1-4.

The first-order logical language over the alphabet $V[V_\nu]$ is $\mathcal{L}[V_\nu] = \mathcal{L}_t[V_\nu] \cup \mathcal{L}_f[V_\nu]$, that is, every word of $\mathcal{L}[V_\nu]$ is either a term or a formula.

The negation, conjunction, disjunction, implication (their meaning is the same as in propositional logic) and quantified formulas are compound formulas.

The prime formulas of the first-order logical language are the atomic formulas and the quantified formulas.

Kinds of variable occurrence: An occurrence of variable x of a formula is:

- free, if it does not fall within the scope of a quantifier relating to x,
- bound, if it falls within the scope of a quantifier relating to x.

Kinds of variable: A variable x of a formula is:

- free, if all of its occurrences are free,
- bound, if all of its occurrences are bound, and
- mixed, if it has both free and bound occurrences.

Closedness, openness of a formula: A formula is:

- closed, if all of its variables are bound,
- open, if at least one of its variables has a free occurrence and
- quantifier-free, if there is no quantifier in it

Remark: closed formulas symbolize first-order statements (a first-order statement is nothing other than a declarative sentence formulated for a set of elements).

1.3.2 The semantics of first-order logic

Mathematical structure: By a mathematical structure we mean a quadruple $< U, R, M, K >$, where:

1. $U = \bigcup_\pi U_\pi$ is a nonempty base set (universe),
2. R is the set of logical functions (relations) defined on U,
3. M is the set of mathematical functions (basic operations) defined on U,
4. K is the set of distinguished elements (constants) of U (can be empty).

Interpretation: The interpretation is a mathematical structure $< U, R, M, K >$ and a function quadruple $\mathcal{I} =< \mathcal{I}_{Srt}, \mathcal{I}_{Pr}, \mathcal{I}_{Fn}, \mathcal{I}_{Cnst} >$, where:

- the function $\mathcal{I}_{Srt} : \pi \mapsto U_\pi$ gives to every sort $\pi \in Srt$ a nonempty set U_π, the set of individuals of sort π,
- the function $\mathcal{I}_{Pr} : P \mapsto P^{\mathcal{I}}$ gives to every predicate symbol $P \in Pr$ of the form $(\pi_1, \pi_2, ..., \pi_k)$ a logical function (relation) $P^{\mathcal{I}} : U_{\pi_1} \times U_{\pi_2} \times ... \times U_{\pi_k} \to \mathbb{L}$,
- the function $\mathcal{I}_{Fn} : f \mapsto f^{\mathcal{I}}$ assigns to every function symbol $f \in Fn$ of the form $(\pi_1, \pi_2, ..., \pi_k, \pi)$ a mathematical function (operation) $P^{\mathcal{I}} : U_{\pi_1} \times U_{\pi_2} \times ... \times U_{\pi_k} \to U_\pi$,
- and $\mathcal{I}_{Cnst} : c \mapsto ct^{\mathcal{I}}$ assigns to every constant symbol $c \in Cnst$ of sort π an individual of the individual domain U_π, that is, $c^{\mathcal{I}} \in U_\pi$.

Variable evaluation: Let $\mathcal{I}$ be an interpretation of the language $\mathcal{L}[V_\nu]$, let the universe of the interpretation be U and let V denote the set of variables of the language. A mapping $\kappa : V \to U$, where if x is a variable of sort π, then $\kappa(x) \in U_\pi$, is called a variable evaluation in $\mathcal{I}$.

Semantics of $\mathcal{L}_t[V_\nu]$: Let $\mathcal{I}$ be an interpretation of the language $\mathcal{L}[V_\nu]$ and κ a variable evaluation in $\mathcal{I}$. The value of a term t of sort π of the language $\mathcal{L}[V_\nu]$ in $\mathcal{I}$ under the variable evaluation κ is the following individual in U_π – denoted by $|t|^{\mathcal{I},\kappa}$:

1. if $c \in Cnst$ is a constant symbol of sort π, then $|c|^{\mathcal{I},\kappa}$ is the individual $c^{\mathcal{I}}$ in U_π,
2. if x is a variable of sort π, then $|x|^{\mathcal{I},\kappa}$ is the individual $\kappa(x)$ in U_π,

3. if $t_1, t_2, ..., t_k$ are terms respectively of sorts $\pi_1, \pi_2, ..., \pi_k$ and their values under the variable evaluation κ are respectively the individuals $|t_1|^{\mathcal{I},\kappa}$ in U_{π_1}, the $|t_2|^{\mathcal{I},\kappa}$ in U_{π_2} ... and $|t_k|^{\mathcal{I},\kappa}$ in U_{π_k}, then for a function symbol $f \in Fn$ of the form $(\pi_1, \pi_2, ..., \pi_k, \pi)$, $|f(t_1, t_2, ..., t_k)|^{\mathcal{I},\kappa}$ is the individual $f^{\mathcal{I}}(|t_1|^{\mathcal{I},\kappa}, |t_2|^{\mathcal{I},\kappa}, ..., |t_k|^{\mathcal{I},\kappa})$ in U_π.

x-variant of a variable evaluation: Let x be a variable. The variable evaluation κ^* is an x-variant of the variable evaluation κ if $\kappa^*(y) = y$ for every variable y different from x.

Logical value of a first-order logical formula: Let $\mathcal{I}$ be an interpretation of the language $\mathcal{L}[V_\nu]$ and κ a variable evaluation in $\mathcal{I}$. To a formula C of the language $\mathcal{L}[V_\nu]$ we assign, in $\mathcal{I}$ under the variable evaluation κ, the following truth value – denoted by $|C|^{\mathcal{I},\kappa}$:

1. $|P(t_1, t_2, ..., t_k)|^{\mathcal{I},\kappa} = \left\{ \begin{array}{ll} true & : P^{\mathcal{I}}(|t_1|^{\mathcal{I},\kappa}, |t_2|^{\mathcal{I},\kappa}, ..., |t_k|^{\mathcal{I},\kappa}) = true \\ false & : otherwise \end{array} \right\}$
2. let $|\neg A|^{\mathcal{I},\kappa}$ be $\neg |A|^{\mathcal{I},\kappa}$
3. let $|A \wedge B|^{\mathcal{I},\kappa}$ be $|A|^{\mathcal{I},\kappa} \wedge |B|^{\mathcal{I},\kappa}$
4. let $|A \vee B|^{\mathcal{I},\kappa}$ be $|A|^{\mathcal{I},\kappa} \vee |B|^{\mathcal{I},\kappa}$
5. let $|A \supset B|^{\mathcal{I},\kappa}$ be $|A|^{\mathcal{I},\kappa} \supset |B|^{\mathcal{I},\kappa}$
6. $|\forall x A|^{\mathcal{I},\kappa} = \left\{ \begin{array}{ll} true & : |A|^{\mathcal{I},\kappa^*} = true \ for every \ x-variant \ \kappa^* \ of \ \kappa \\ false & : otherwise \end{array} \right\}$
7. $|\exists x A|^{\mathcal{I},\kappa} = \left\{ \begin{array}{ll} true & : |A|^{\mathcal{I},\kappa^*} = true \ for some \ x-variant \ \kappa^* \ of \ \kappa \\ false & : otherwise \end{array} \right\}$

Satisfiability of a first-order formula: A first-order formula A is satisfiable if there is an interpretation $\mathcal{I}$ and a variable evaluation κ for which $|A|^{\mathcal{I},\kappa} = true$ (in which case we say that the interpretation $\mathcal{I}$ and variable evaluation κ satisfy A), otherwise unsatisfiable.

If the formula A is closed, its truth value is determined solely by the interpretation. If $|A|^{\mathcal{I}} = true$, we say that $\mathcal{I}$ satisfies A or otherwise: $\mathcal{I}$ is a model of A ($\mathcal{I} \models A$).

Logically true first-order formula: A first-order logical formula A is logically true if in every interpretation $\mathcal{I}$ and under every variable evaluation κ of $\mathcal{I}$, $|A|^{\mathcal{I},\kappa} = true$. Its notation: $\models A$.

Semantic consequence: We say that the formula G is a *semantic consequence* of the formula set $\mathcal{F}$ if for every interpretation $\mathcal{I}$ for which $\mathcal{I} \models \mathcal{F}$ holds, $\mathcal{I} \models G$ is also true (notation: $\mathcal{F} \models G$).

Theorem: Let $A_1, A_2, ..., A_n, B$ ($n \geq 1$) be arbitrary formulas from the same first-order logical language. Then $\{A_1, A_2, ..., A_n\} \models B$ if and only if $A_1 \wedge A_2 \wedge ... \wedge A_n \wedge \neg B$ is unsatisfiable.

Resolution: Resolution can also be carried out in first-order predicate calculus, and moreover the method is sound and complete. Difficulty can be caused by the construction of the clauses, which consist of the conjunction of closed, universally quantified literals. For this our tools are the prenex and Skolem forms.